SQL & PostgreSQL Interview Knowledge

Practical SQL, database design and PostgreSQL reference

Purpose
SQL fundamentals, relational design, performance, transactions, PostgreSQL, SQLAlchemy and security.

Audience: Final-year engineering students and fresher candidates targeting Python, software development, ML, AI engineering and GenAI roles.

RAG design: Content is explicit and retrieval-friendly so chunks can stand alone. Source identifiers, definitions, examples, troubleshooting and interview reasoning are intentionally separated.

Table of Contents

1. 1. Relational model and CRUD
2. 2. Filtering, grouping and NULL
3. 3. Joins and subqueries
4. 4. Keys, constraints and normalization
5. 5. Indexes and performance
6. 6. Transactions and ACID
7. 7. PostgreSQL types and practical SQL
8. 8. SQLAlchemy and migrations
9. 9. Security and troubleshooting

1. Relational model and CRUD

A relational database stores rows in tables with defined columns. SQL describes the desired result; the optimizer chooses how to execute it. CRUD corresponds to INSERT, SELECT, UPDATE and DELETE.

Schema constraints such as PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL and CHECK express data rules near the data, reducing the chance that every application path must reimplement them.

```
CREATE TABLE users (
    id BIGSERIAL PRIMARY KEY,
    email TEXT NOT NULL UNIQUE,
    name TEXT NOT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

SELECT id, name FROM users WHERE email = :email;
```

2. Filtering, grouping and NULL

WHERE filters rows before grouping. GROUP BY forms groups for aggregate functions. HAVING filters groups after aggregation. ORDER BY controls output ordering.

NULL means unknown/missing. `column = NULL` is not a true condition; use `IS NULL`. `COALESCE` can replace NULL with a chosen fallback.

```
SELECT department, COUNT(*) AS employee_count, AVG(salary) AS avg_salary
FROM employees
WHERE active = true
GROUP BY department
HAVING COUNT(*) >= 5
ORDER BY avg_salary DESC;
```

3. Joins and subqueries

INNER JOIN returns matching rows. LEFT JOIN preserves rows from the left table even when no match exists. A common mistake is placing a right-table filter in WHERE after a LEFT JOIN, which can remove NULL-extended rows and change the effective behavior.

CTEs introduced with WITH can make multi-step queries easier to read. Subqueries can be correlated or independent; choose the clearest query shape and inspect the plan for performance.

Join	Result	Typical use
INNER	Only matches	Require relationship
LEFT	All left + matches	Find optional/missing relationships
Self	Table joined to itself	Hierarchy/comparison
CTE	Named subquery	Readable multi-step logic

```
SELECT u.id, u.name, o.total
FROM users AS u
LEFT JOIN orders AS o
  ON o.user_id = u.id
 AND o.status = 'paid';

WITH totals AS (
  SELECT user_id, SUM(total) total_spend
  FROM orders GROUP BY user_id
)
SELECT u.name, totals.total_spend
FROM users u JOIN totals ON totals.user_id = u.id;
```

4. Keys, constraints and normalization

A primary key identifies a row. A foreign key references a key in another table and enforces referential integrity. UNIQUE prevents duplicate values for a constrained key or combination.

Normalization reduces unnecessary duplication and update anomalies. First/second/third normal form are useful interview concepts, but real schemas may intentionally denormalize for measured read

performance or reporting.

Constraint	Purpose
PRIMARY KEY	Row identity
FOREIGN KEY	Valid relationships
UNIQUE	No duplicate constrained value
NOT NULL	Value required
CHECK	Domain/business rule

5. Indexes and performance

Indexes are auxiliary structures that can speed reads but consume storage and add write/maintenance work. PostgreSQL commonly uses B-tree indexes for equality and range operations. Other index types target specialized workloads.

Index design should follow real query patterns. Composite index column order matters. Use EXPLAIN/EXPLAIN ANALYZE to inspect plans rather than guessing.

```
CREATE INDEX idx_orders_user_created
ON orders (user_id, created_at DESC);

EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM orders
WHERE user_id = 42
ORDER BY created_at DESC
LIMIT 20;
```

6. Transactions and ACID

Atomicity means a transaction behaves as a unit. Consistency means database rules remain satisfied. Isolation controls interactions among concurrent transactions. Durability means committed changes survive failures according to database guarantees.

Application correctness may still require locks, constraints, isolation choices or idempotency. ACID does not automatically prevent every business-level race condition.

```
BEGIN;
UPDATE accounts SET balance = balance - 500 WHERE id = 1;
UPDATE accounts SET balance = balance + 500 WHERE id = 2;
COMMIT;
```

7. PostgreSQL types and practical SQL

PostgreSQL provides UUID, JSONB, arrays, timestamps, numeric types and more. JSONB is useful for semi-structured fields that still need querying, but it should not automatically replace relational columns for stable business entities.

Use timezone-aware timestamps such as TIMESTAMPTZ when representing instants, and make timezone assumptions explicit at application boundaries.

```
CREATE TABLE events (
    id UUID PRIMARY KEY,
    user_id BIGINT NOT NULL REFERENCES users(id),
    payload JSONB NOT NULL,
    occurred_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

SELECT payload->>'event_type', COUNT(*)
FROM events GROUP BY payload->>'event_type';
```

8. SQLAlchemy and migrations

SQLAlchemy can be used through its Core expression system or ORM. A session should have a clear lifecycle and should not be casually shared across unrelated concurrent requests.

Schema migrations, commonly managed with Alembic in SQLAlchemy projects, record controlled changes. Production migrations should be reviewed for locking, backfill and compatibility concerns.

```
from sqlalchemy import create_engine, text
engine = create_engine(DATABASE_URL, pool_pre_ping=True)
with engine.begin() as conn:
    rows = conn.execute(
        text("SELECT id, name FROM users WHERE active = :active"),
        {"active": True},
    ).all()
```

9. Security and troubleshooting

Parameterized queries keep values separate from SQL structure and are a primary SQL-injection defense. Never concatenate untrusted text into SQL syntax.

When a database connection fails, check the configured URL, hostname/DNS, credentials, TLS/SSL requirements, network policy, pool limits and database availability. When a query is slow, inspect the actual plan and data distribution.

- Do not expose database passwords in logs.
- Do not rely only on application-level uniqueness checks; use database constraints for invariants.

Interview question

WHERE versus HAVING?

Strong answer: WHERE filters rows before grouping; HAVING filters groups after aggregation. A condition involving COUNT usually belongs in HAVING.

Follow-up: Can HAVING be used without GROUP BY?

Interview question

Why can an index make writes slower?

Strong answer: Every relevant INSERT/UPDATE/DELETE may also need to maintain index entries. Indexes use storage and memory/cache resources. The trade-off is worthwhile when the read benefit matches the workload.

Follow-up: How do you decide which columns to index?

Interview question

What is normalization?

Strong answer: A design approach that reduces unnecessary duplication and update anomalies by separating facts into related tables with appropriate keys.

Follow-up: When would you intentionally denormalize?

Interview question

What is SQL injection?

Strong answer: An attack where untrusted input changes the intended SQL structure. Parameterized statements prevent ordinary values from being interpreted as SQL syntax.

Follow-up: Why is escaping alone not the preferred defense?

Interview question

Why use transactions for money movement?

Strong answer: The related updates must succeed or fail together, and concurrent operations must preserve business invariants. A transaction plus appropriate constraints/locking prevents partial transfers.

Follow-up: What if two withdrawals race?

Scenario: query became slow after growth

Situation: A query was fast with 1,000 rows and is now slow with millions.

Reasoning: Run EXPLAIN ANALYZE, inspect scans/sorts/joins, check statistics and indexes, reduce unnecessary rows/columns and confirm the query shape matches the workload. Avoid adding indexes blindly.

Scenario: duplicate order on retry

Situation: Two requests can create the same logical order.

Reasoning: Define an idempotency/business key and enforce it with a unique constraint. Handle a duplicate-key result as a normal concurrency outcome rather than relying on SELECT-then-INSERT.

Interview question

What is ACID?

Strong answer: Atomicity, Consistency, Isolation and Durability. Explain each in the context of a transaction, then add that application-level correctness can require constraints and concurrency control.

Follow-up: How does isolation relate to race conditions?

Revision checklist

- Write safe CRUD queries.
- Handle NULL correctly.
- Choose INNER versus LEFT JOIN.
- Explain GROUP BY/HAVING.
- Explain keys and constraints.
- Explain normalization and measured denormalization.
- Read a basic EXPLAIN plan.
- Explain ACID and transaction boundaries.
- Use parameterized SQL.
- Understand SQLAlchemy sessions and migrations.

Reference notes

Primary documentation used for version-sensitive guidance. Verify exact package/API behavior against the versions installed in your project.

https://www.postgresql.org/docs/

https://docs.sqlalchemy.org/

https://alembic.sqlalchemy.org/

End of knowledge base document